Top 8 TinyML Frameworks and Compatible Hardware Platforms (TensorFlow Lite, Edge Impulse, PyTorch Mobile, etc.)

DFRobot Jul 12 2024 373683

TinyML frameworks provide a robust and efficient infrastructure that enables organizations and developers to harness their data and deploy advanced algorithms on edge devices effectively. These frameworks offer a wide range of tools and resources specifically designed to drive strategic initiatives in Tiny Machine Learning. This article highlights the top 8 well-known frameworks for TinyML implementation, such as TensorFlow Lite (TF Lite), Edge Impulse, PyTorch Mobile, uTensor, and platforms like STM32Cube.AI, NanoEdgeAIStudio, NXP eIQ, and Microsoft's Embedded Learning Library. It also outlines the compatible hardware platforms and target applications for these frameworks, assisting users in quickly identifying the most suitable TinyML frameworks.

What is TinyML?

TinyML is a branch of machine learning that focuses on creating and implementing machine learning models on low-power, small-footprint microcontrollers such as the Arduino.Machine Learning models require a significant amount of computing power. It can't be used to make models on devices that run on batteries, tiny Machine Learning (tinyML) is used in this case.

Why do we need TinyML?

TinyML enables the execution of machine learning models on small microcontrollers, including devices like Raspberry Pi and ESP32. While these devices are impressive, even the smallest Raspberry Pi consumes hundreds of milliwatts of power, similar to a main CPU.

tinyML can be implemented on:

- The compact and low-cost devices (microcontrollers) very low power usage (tinyML consumes less than one milliwatt of power)
- Memory capacity is limited

- Low lag time (almost immediate) integrated machine learning algorithms analysis.

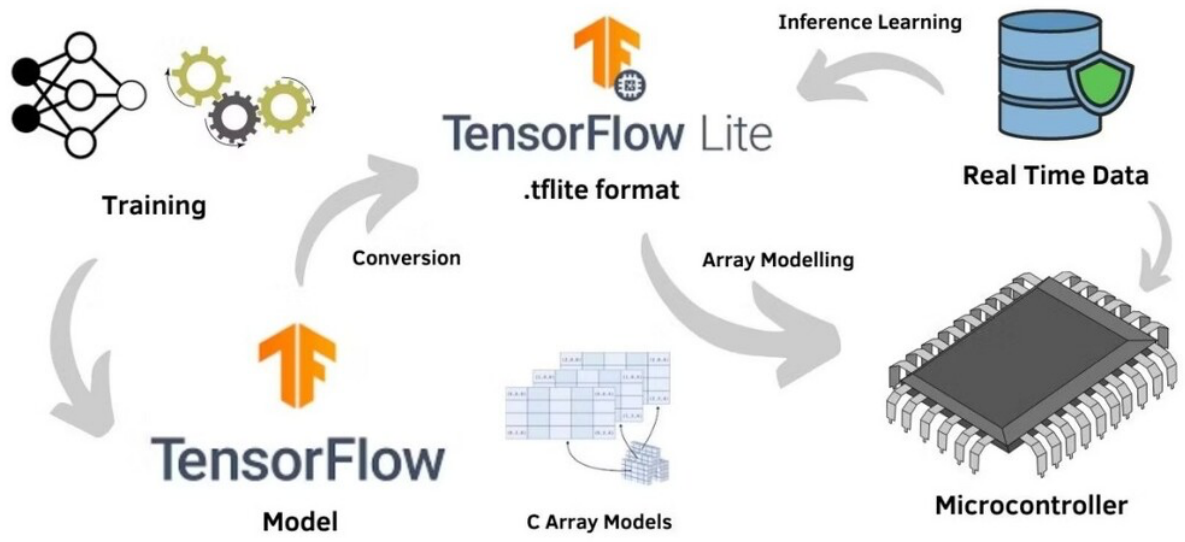
What is a TinyML Framework?

A TinyML framework refers to a specialized software or tool that enables developers and engineers to train machine learning models specifically designed for deployment on edge devices and embedded systems. These platforms provide the necessary infrastructure, algorithms, and resources to facilitate the training process of Tiny Machine Learning (TinyML) models, which are optimized to run on resource-constrained devices with low power consumption. TinyML frameworks typically support tasks such as data collection, model training, optimization, and deployment on edge devices, allowing for the development of efficient and effective machine learning models tailored for edge computing environments.

8 TinyML Frameworks

1. TensorFlow Lite

TensorFlow is Google's open-source machine learning framework which helps in developing machine learning models quickly. For TinyML, there is TensorFlow Lite Micro, which is a specialised version of TensorFlow for microcontrollers and other devices with only a few kilobytes of memory. The core runtime just fits in 16 KB on an Arm Cortex M3 and can run many basic models. It doesn't require operating system support, any standard C or C++ libraries, or dynamic memory allocation. It is mostly compatible with Arm Cortex-M Series processors. An ESP32 port is available as well.



TensorFlow Lite

Key advantages

- **Fast Inference:** It enables rapid processing on mobile devices by leveraging hardware accelerators like GPU and DSP. This leads to quick and efficient inference for real-time applications such as object detection and gesture recognition.
- **Flexibility:** This format supports Android, iOS, Linux, and microcontrollers, making it versatile for different devices. For details hardware platforms supported, see Table One: Hardware Platforms for 8 TinyML Frameworks.
- **Ease of Integration:** TensorFlow Lite seamlessly integrates with existing TensorFlow workflows, allowing developers to train models using TensorFlow's tools and libraries, and then convert them to the TensorFlow Lite format for mobile and embedded deployment. TensorFlow Lite also offers APIs compatible with popular programming languages like Python, Java, and C++, enabling easy integration of machine learning capabilities into mobile and embedded applications.

Limitations

TensorFlow Lite for Microcontrollers is designed for the specific constraints of microcontroller development. If you are working on more powerful devices (for example, an embedded Linux device like the Raspberry Pi), the standard TensorFlow Lite framework might be easier to integrate.
The following limitations should be considered:

- Support for a limited subset of TensorFlow operations
- Support for a limited set of devices
- Low-level C++ API requiring manual memory management
- On-device training is not supported

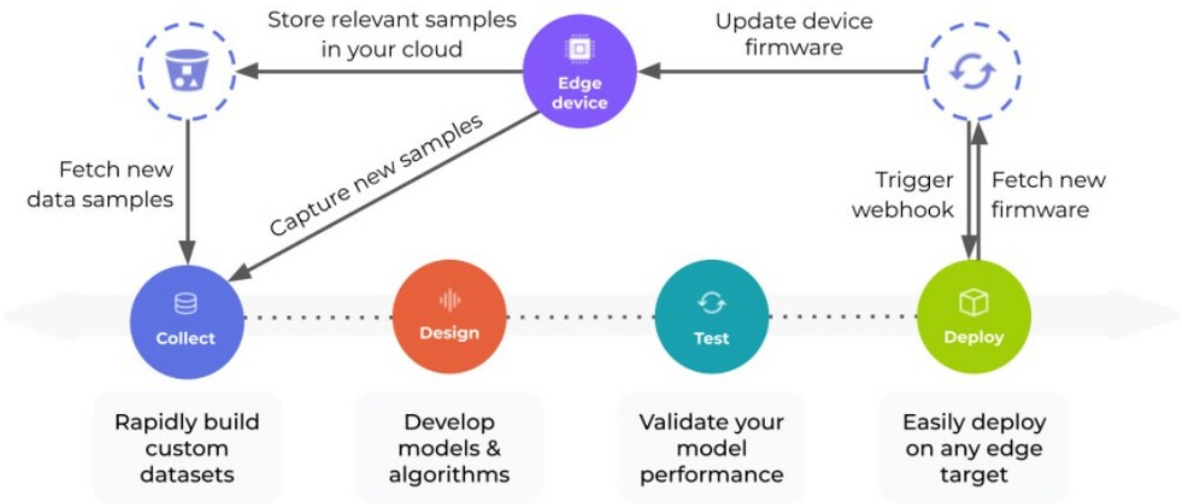
2. Edge Impulse

Edge Impulse offers the latest in machine learning tooling, enabling all enterprises to build smarter edge products. Edge Impulse is undoubtedly the easiest way for anybody to collect data, train a model, and deploy it on a microcontroller.

Key Advantages of Edge Impulse

Edge AI lifecycle

Edge Impulse helps with every step along the edge AI lifecycle, from collecting data, extracting features, designing machine learning (ML) models, training and testing those models, and deploying the models to end devices. Edge Impulse easily plugs into other machine-learning frameworks so that you can scale and customize your model or pipeline as needed.



Edge Optimized Neural (EON™) Compiler

Optimized Neural (EON™) Compiler, which lets you run neural networks in 25-55% less RAM, and up to 35% less flash, while retaining the same accuracy, compared to TensorFlow Lite for Microcontrollers.

Here's an example of the difference EON makes on a typical model in Edge Impulse. Below you'll see the time per inference, RAM, and ROM usage of a keyword spotting model with a 2D convolutional neural network,

running on a Cortex-M4F. At the top: EON, at the bottom: the same model running using TensorFlow Lite for Microcontrollers.

2D Convolutional Neural Network running under EON

Edge Impulse Limitations

Compatibility Issues

There are some limitations in terms of certain advanced customization options, compatibility with specific hardware, or the learning curve for those who are new to machine learning and IoT technologies.

Limited Customization

The platform might feel somewhat limited in terms of building very complex or specialized models. Users with advanced machine learning needs might desire more extensive customization options or support for more advanced model architectures.

3. PyTorch Mobile

It belongs to the PyTorch ecosystem that aims to support all phases starting from training to deployment of machine learning models to smart phones (e.g., Android, iOS). Several APIs are available to preprocess machine learning in mobile applications (PyTorch, 2021). It can support the scripting and tracing of TorchScript IR. Further support is given for the XNNPACK 8-bit quantized kernel targeting ARM CPUs. It can also support GPUs, digital signal processors, and neural processing units. Optimization facility for mobile phone deployment is paved via the mobile interpreter. Currently, it supports image segmentation, object detection, video processing, speech recognition, and question answering tasks.

Key features:

- 1. Multi-platform support: PyTorch Mobile can run on iOS, Android, and Linux, providing developers with a wide range of deployment options.
- 2. API availability: It offers APIs covering common preprocessing and integration tasks, facilitating the integration of machine learning into mobile applications.
- 3. TorchScript support: It supports both tracing and scripting through TorchScript IR to meet different deployment needs.

PyTorch vs. TensorFlow

- 1. Both PyTorch and TensorFlow achieve similar accuracy, but TensorFlow requires much longer training time while using lower memory.
- 2. PyTorch allows for faster prototyping compared to TensorFlow. However, if custom functionality is needed within a neural network, TensorFlow may be a better choice.
- 3. TensorFlow treats neural networks as static objects; if you want to change the behavior of your model, you have to start from scratch. With PyTorch, you can dynamically adjust the neural network at runtime, making it easier to optimize the model.
- 4. Effective debugging with TensorFlow requires a special debugger tool that allows inspection of how network nodes are computed at each step. PyTorch can be debugged using many widely available Python debugging tools.
- 5. PyTorch and TensorFlow both offer methods to accelerate model development and reduce boilerplate code. However, the core difference between PyTorch and TensorFlow is that PyTorch is more "pythonic" and based on an object-oriented approach, while TensorFlow provides more options for flexibility overall.

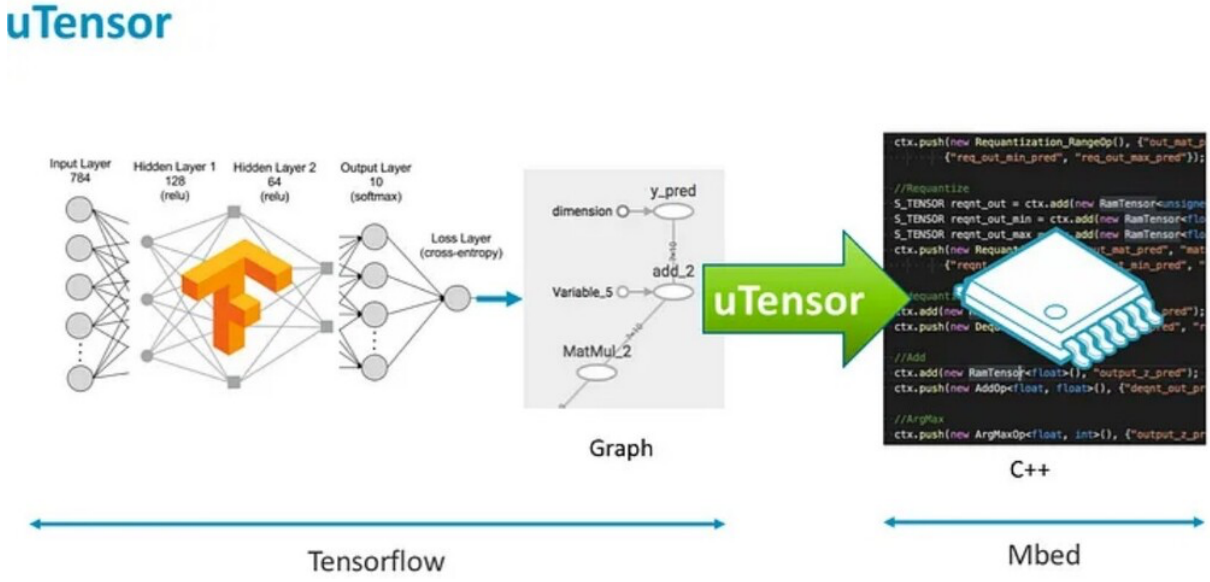
4. uTensor

uTensor is a lightweight machine learning inference framework optimized for Arm platforms and based on TensorFlow.It takes a neural network model by using Keras for training. It then converts the trained model into a C++. The uTensor helps to convert the model for suitable deployment in the Mbed, ST, and K64 boards. The uTensor is a small size module that

requires only 2 KB on disk. A Python SDK is used to customize the uTensor from ground up. It depends on the following tool sets such as, Python, uTensor-CLI, Jupyter, Mbed-CLI, and ST-link (for ST boards). Initially, a model is created and then defined with a quantization effect. The next step is code generation for suitable edge devices.

Module	.text
uTensor/src/uTensor/core	1275(+1275)
uTensor/src/uTensor/tensors	791(+791)

In TensorFlow, a model is built and trained in a flow. uTensor acquires the model and generates a .cpp and .hpp file. These files contain generated C++11 code for inference. Using uTensor on the embedded side is as simple as copy-and-paste.



Key Features:

- **Secure and Reliable:** By managing metadata and actual data in dedicated memory regions, uTensor ensures that memory usage stays within predefined limits and provides compile-time error checking.
- **User-Friendly API:** With a high-level language style interface, uTensor simplifies the development process while supporting optimization of performance directly at the C++ level.

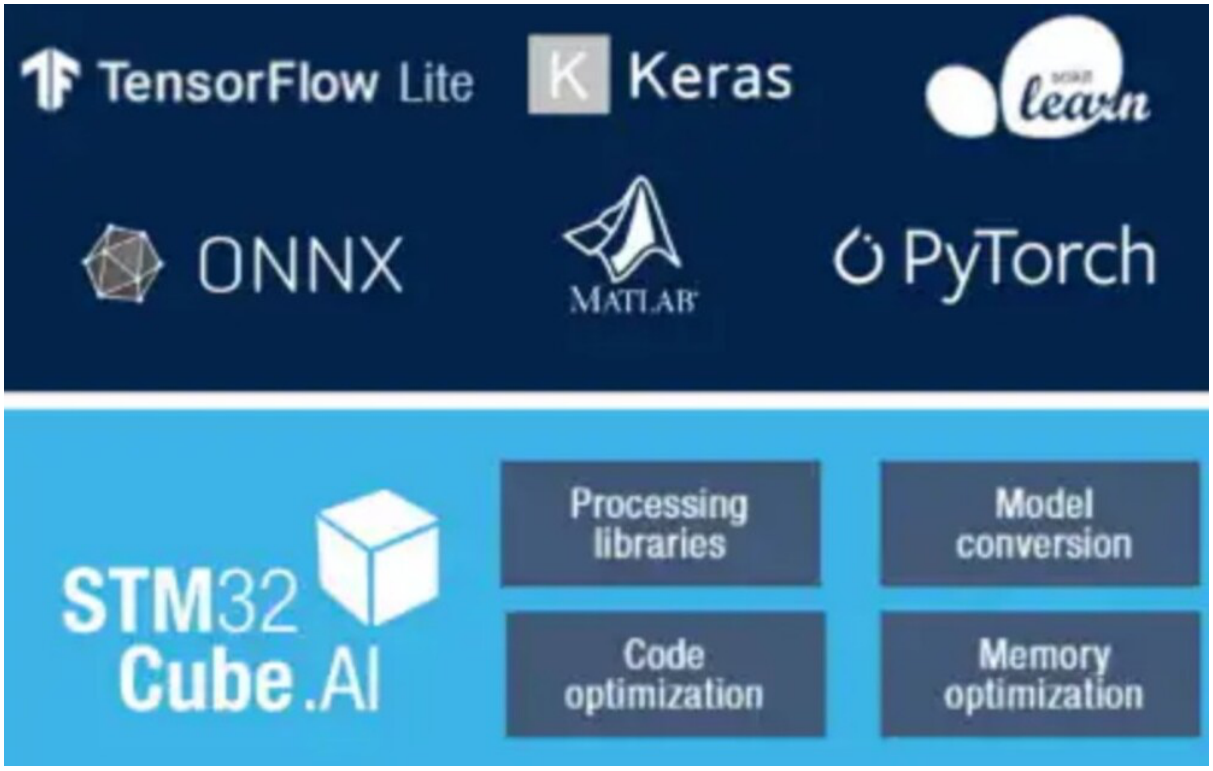
- **Flexible Extensibility:** From core library to default implementations, uTensor allows for customization and optimization, including tensor implementations, operators, memory allocators, and more

5. STM32Cube.AI

It is a code generation and optimization software that makes machine learning and AI related tasks easier for STM32 ARM Cortex M−based boards. Implementation of neural networks in STM32 board can be directly achieved by using STM32Cube.AI to convert the neural nets into an optimized code for the most appropriate MCU. It can optimize the memory usage during run time. It can use any trained model by conventional tools such as TFL, ONNX, Matlab, and PyTorch. This tool is actually an extension of the original STM32CubeMX framework that helps STM32Cube.AI to perform code generation for target STM32 edge device and middleware parameter estimation.

Key Features:

- Allows you to generate a library from pre-trained Neural Networks and typical Machine Learning models, which is optimized for STM32
- Supports the most popular frameworks (Tensor Flow Lite, Keras, qKeras, Pytorch, ONNX and more)
- Easy portability across different STM32 microcontroller series through STM32Cube integration
- Offers the possibility to integrate converted Neural Networks libraries more easily, thanks to application-oriented code examples (function packs)



STM32Cube.AI

6. NanoEdgeAIStudio

NanoEdgeAIStudio is an automated machine learning tool designed for STM32 developers. This tool does not require users to have specialized data science skills or expertise in the field of artificial intelligence (AI) as it offers a user-friendly environment and supports all STM32 products. The data logging feature of NanoEdge AI Studio helps you collect and manage high-speed data from industrial-grade sensors without the need to write any code for processing industrial-grade sensors. NanoEdge AI Studio also offers features such as auto-search engines, anomaly detection, classification, regression algorithms, making machine learning on edge devices more accessible.

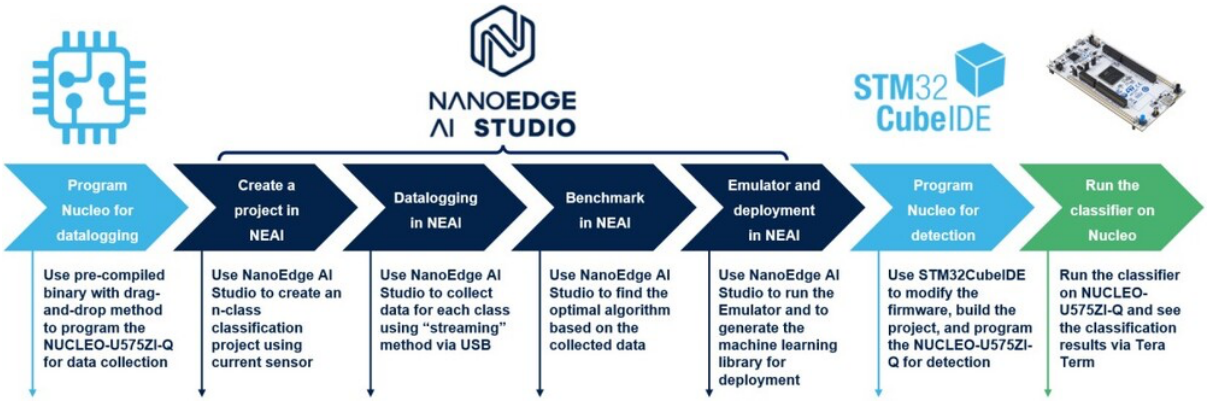
STM32Cube.AI vs NanoEdgeAIStudio

STM32Cube.AI	STM32Cube.AI vs NanoEdgeA
Cube.AI is a tool that can rapidly evaluate, convert, and deploy machine learning or deep neural networks on STM32 MCUs. In other words, its input is pre-trained neural network models or machine learning models, and its output is code that can run on STM32.	The NanoEdge AI software tool learning solution for embedded data collection, model selection and optimized deployment, it is to-end machine learning optimiz

Cube.AI supports all mainstream AI frameworks such as TensorFlow Lite, Carrots, PyTorch, ONNX, and several machine learning algorithms.

Its advantage is that it does not
has relatively high memory effic

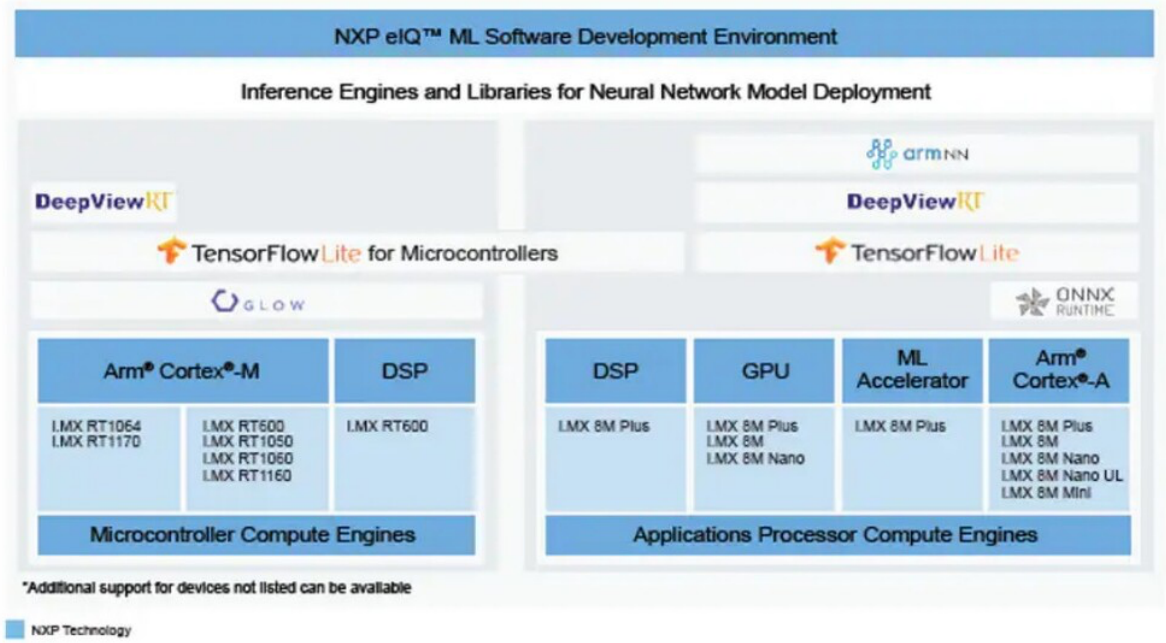
STM32Cube.AI vs NanoEdgeAIStudio



NanoEdgeAIStudio

7. NXP eIQ® Machine Learning Software Development Environment

NXP Semiconductors eIQ Machine Learning Software Development Environment is a combination of libraries and development tools for use with NXP microprocessors and microcontrollers. The eIQ Machine Learning Software includes the DeepViewRT™ proprietary inference engine. The software makes inferences from neural network (NN) artificial intelligence (AI) models on embedded systems. eIQ Machine Learning (ML) Software offers the key ingredients to deploy various ML algorithms at the edge (eIQ = edge intelligence) and includes inference engines, NN compilers, vision and sensor solutions, and hardware abstraction layers. Four main inference engines & libraries – OpenCV, Arm® NN, Arm CMSIS-NN, TensorFlow Lite, and proprietary DeepViewRT runtime inference are supported.



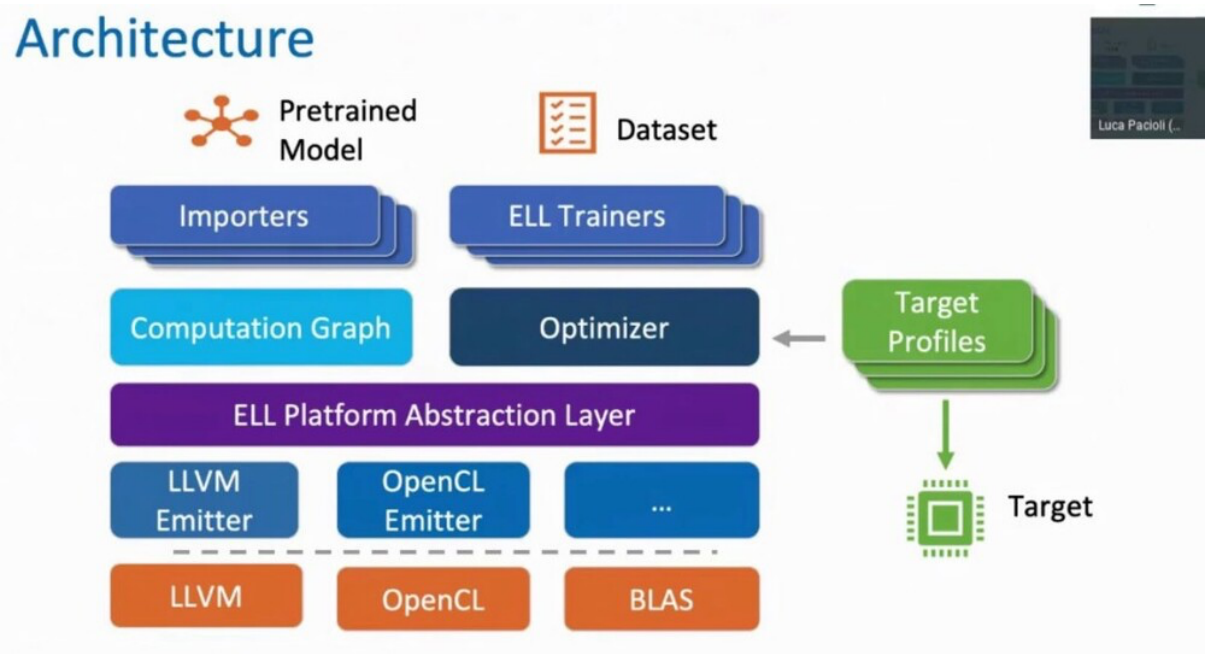
NXP eIQ® Machine Learning Software Development Environment

Key Features:

- Extensive optimization tools and Model Zoo
- Large choice of inference engines, including auto quality option, with a common API
- Run ML workloads on multiple processor cores

8. Embedded Learning Library (ELL)

Microsoft has developed the ELL for supporting TinyML ecosystem for embedded learning. It provides support for Raspberry Pi, Arduino, and micro:bit platforms. The models which are deployed in such devices are internet agnostic, thus no cloud access is required. It supports the image and audio classification at the moment. The library also provides a set of software tools and an optional interface in Python, written in modern C++



Microsoft Embedded Learning Library (ELL)

Compatible Hardware Platforms for 8 TinyML Frameworks

You may be interested in which hardware platforms the 8 TinyML training platforms supported. The following table lists the details of the available hardware platforms supporting the design environment i.e., frameworks/libraries.

TinyML Platforms	Hardware Platforms
TensorFlow Lite (TFL)	Arduino Nano 33 BLESense, Sparkfun Edge, STM32F746 Discovery Kit TensorFlow Lite for Microcontrollers Kit, Adafruit Circuit Playground Blue Espressif ESP-EYE, Wio Terminal: ATSAMD51, Himax WE-I Plus EVB E Synopsys Designware ARC EM Software Development Platform, Sony S
Edge Impulse	Arduino Nano 33 BLE Sense, Arduino Nicla Sense ME, Arduino Nicla Vi Shield, Espressif ESP32, Himax WE-I Plus, Nordic Semi Nrf52840 DK, N
Utensor	Mbed, ST K64 ARM Boards
Pytorch Mobile	NNAPI (Android), Coreml (iOS), Metal GPU (iOS), Vulkan (Android)
Nanoedge AI Studio	STM32 Boards
STM32Cube.AI	STM32 ARM CORTEX Boards
Embedded Learning Library (ELL)	Raspberry Pi, Arduino, micro:bit

Table One: Hardware Platforms for 8 TinyML Frameworks

Targeted Applications of 8 TinyML Frameworks

TinyML Platforms	Target Applications
TensorFlow Lite (TFL)	Image and Audio Classification, Object Detection, Pose Estimation, Sp Segmentation, Video Classification, Text Classification, Reinforcement Character Recognition
Edge Impulse	Asset Tracking and Monitoring, Human Interfaces, Predictive Maintena
Utensor	Image Classification, Gesture Recognition, Acoustic Detection and Mo
Pytorch Mobile	Computer Vision and Natural Language Processing
Nanoedge AI Studio	Anomaly Detection, Predictive Maintenance, Condition Monitoring, Ass Recognition
STM32Cube.AI	Anomaly Detection, Predictive Maintenance, Condition Monitoring, Ass Recognition
Embedded Learning Library (ELL)	Image And Audio Classification

Table Two: Hardware Platforms for 8 TinyML Frameworks

Conclusion

This article explores 8 best-known TinyML frameworks, detailing their key features and limitations. Among these platforms, TensorFlow stands out for its exceptional flexibility, supporting over 20 hardware platforms such as Arduino Nano, Sparkfun Edge, and STM32F746 Discovery Kit. It is suitable for a range of target applications including Image and Audio Classification, Object Detection, Pose Estimation, Speech, and Gesture Recognition. Edge Impulse introduces the Edge Optimized Neural (EON™) Compiler, capable of reducing neural network RAM usage by 25-55% and flash storage by up to 35%. PyTorch offers faster prototyping compared to TensorFlow, while uTensor is a compact module requiring only 2 KB of disk space. Additionally, industry leaders ST, NXP, and Microsoft have also introduced their TinyML implementation platforms, further advancing the development of TinyML technology.